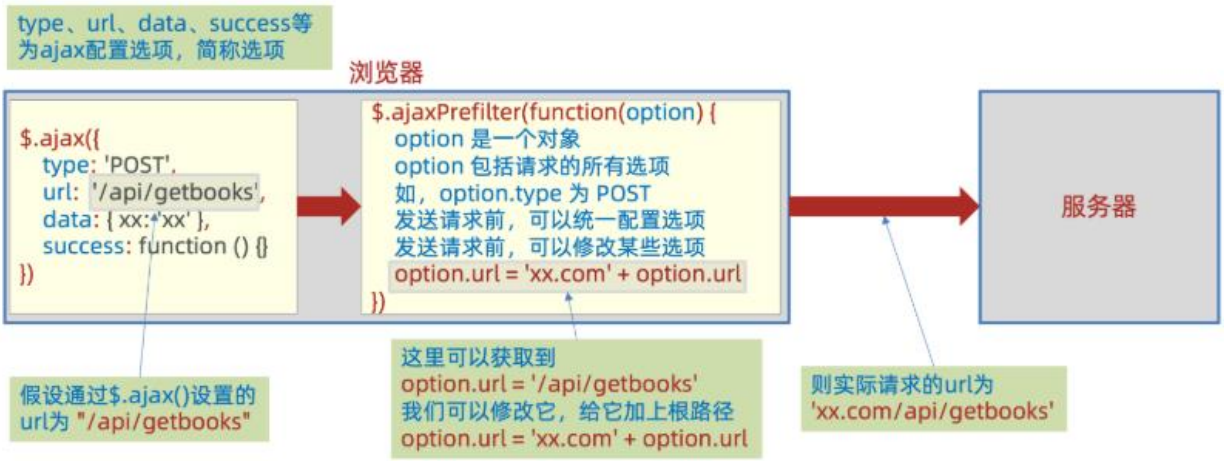
- **$.ajaxPrefilter()**
  - 统一配置或修改ajax选项
  - 必须写在要修改的ajax请求之前



- **jQuery方法请求参数的本质**
  - 无论使用 `$.get()` 还是 `$.post()` 还是 `$.ajax()` 方法，都可以设置请求参数，即 `data`。示例如下
    - $.get('url', `data`, function (res) { ... })
    - $.post('url', `data`, function (res) { ... })
    - $.ajax({ `data`: { id: 1 } })
  - 实际上，在使用 jQuery 的上述三个方法的前提下，我们不但可以使用对象形式的参数，也能使用数组或者查询字符串，示例如下：

```
// 这里使用 $.ajax() 举例，另外两个方法同理
$.ajax({
    url: 'http://www.liulongbin.top:3006/api/getbooks',

    // 对象形式的写法
    data: { id: 1, bookname: '西游记' }

    // 数组形式的写法，注意，只能是这种写法
    data: [{name:'id',value:1}, {name:'bookname',value:'西游记'}]

    // 字符串写法，注意，这种类型的字符串，叫做查询字符串
    data: 'id=1&bookname=西游记'
})
```

  - 无论我们填写的何种形式的参数，都会被jQuery转换成查询字符串形式传递到服务器，因为底层支持查询字符串形式的参数，而不支持字面量对象和数组形式。这一特点，可以通过network工具查看。

- **关于HTML表单的补充——form标签**

- form标签的action属性表示提交地址，默认为空，表示提交到当前页面。
- 还有 method、enctype、target 三个属性，了解即可。

- **关于HTML表单的补充——表单域**
  - 单行文本域 `<input type="text" />`
  - 密码框 `<input type="password" />`
  - 单选按钮 `<input type="radio" />`
  - 复选按钮 `<input type="checkbox" />`
  - 隐藏域 `<input type="hidden" />`
  - 文件域 `<input type="file" />`
  - 下拉选择框 `<select><option>xxx</option></select>`
  - 多行文本框 `<textarea></textarea>`

- **关于HTML表单的补充——按钮**
  - 普通按钮（点击之后，默认不会发生任何事）
    - `<button type="button">普通按钮1</button>`
    - `<input type="button" value="普通按钮2" />`
  - 重置按钮（点击之后，默认会重置表单）
    - `<button type="reset">重置按钮1</button>`
    - `<input type="reset" value="重置按钮2" />`
  - 提交按钮（点击之后，默认会提交表单）
    - `<button type="submit">提交按钮1</button>`
    - `<input type="submit" value="提交按钮2" />`
    - <button>提交按钮3</button>

- **serialize和serializeArray方法**
  - jQuery提供的 `serialize()` 或者 `serializeArray()` 方法可以获取表单各项的值
  - 语法：

```
$('form').serialize();
$('form').serializeArray();
```

  - 也就是说，使用 `serialize()` 或者 `serializeArray()` 方法是通过 表单(form) 调用的，所以必须在HTML页面中加入 `<form>...</form>` 标签。如下所示：

```
<form>
    <!-- 把所有的框框、按钮都放这里面 -->
```

```

    <input type="text" name="bookname" /><br />

    <input type="password" name="pwd" /><br />

    <input type="radio" name="sex" value="nan" checked />男
    <input type="radio" name="sex" value="nv" />女<br />

    <select name="address">
        <option value="bj">北京</option>
        <option value="sh">上海</option>
    </select><br />

    <button>提交</button>
</form>
```

- 接下来，就可以使用 `serialize()` 或者 `serializeArray()` 方法 获取全部的值了，代码如下：

```
// 监听表单的 submit 事件 （表单提交时触发）
$('form').on('submit', function (e) {
    // 一定阻止表单提交，否则页面会跳转；默认跳转到当前页面，感觉和刷新一样
    e.preventDefault();
    // 保证页面不会跳转，接下来使用 serialize 获取表单数据
    var str = $(this).serialize();
    var arr = $(this).serializeArray();
});
```

- 小结：
  - 在必须具有 `<form>...</form>` 标签的前提下，才能使用 `serialize()` 或者 `serializeArray()` 方法
  - 各项表单元素（input、select、textarea）必须具备 `name` 属性。
  - 通过`serialize()` 得到的是查询字符串类型；通过 `serializeArray()`得到的是数组类型。结果都可以直接当做Ajax请求的参数。
  - 两个方法均不能获取 禁用状态（disabled）的输入框的值。
  - 两个方法均不能获取文件域（`<input type="file" />`）的值。
  - 两个方法都能获取隐藏域（`<input type="hidden" />`）的值。

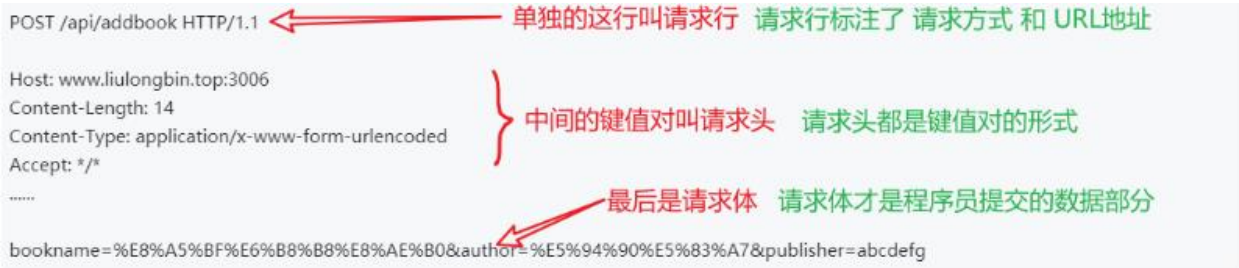
- **了解HTTP协议**
  - 了解HTTP
    - HTTP的全称是超文本传输协议（Hyper Text Transfer Protocol）。

        - 超文本（文本、图像、音频、视频等等）
        - 传输（客户端提交数据到服务器； 服务器返回数据到客户端）
        - 协议（只要叫做协议的，至少涉及双方。这里当然指客户端和服务器“签订”的协议）
    - 就像你在生活中见到的协议一样，协议中规定了双方的权利和义务。
    - HTTP中规定了客户端和服务器双方传输数据的各项要求。
    - 无论是请求，还是响应，其实都有三个部分组成：
        - 请求行 | 响应行
        - 请求头 | 响应头
        - 请求体 | 响应体
  - 请求参数和请求体
    - 当发送请求到服务器的时候，就一次请求来说，实际上客户端向服务器发送的数据如下

```
POST /api/addbook HTTP/1.1

Host: www.liulongbin.top:3006
Content-Length: 14
Content-Type: application/x-www-form-urlencoded
Accept: */*
......

bookname=%E8%A5%BF%E6%B8%B8%E8%AE%B0&author=%E5%94%90%E5%83%A7&publisher=abcdefg
```



    - 拿添加图书举例，请求方式、url、请求体是人为通过代码完成的，其他都是浏览器（偷偷的）设置的，我们无需特别关心。
    - 重要的是，GET方式没有请求体，GET方式的传参是拼接到URL后面的。POST方式是以请求体的方式单独传输数据的。
    - 具体来说，请求方式和请求参数的对应关系如下：

请求方式	URL参数	请求体
GET	√	×
POST	√（不常用）	√（常用）
PUT	√（不常用）	√（常用）
DELETE	√（常用）	√（不常用）

- **XMLHttpRequest对象简介**

- 纲要
  - `XMLHttpRequest` 是一个内建的浏览器对象，Ajax技术的核心就是`XMLHttpRequest` 对象。
  - jQuery中的 `$.ajax()` 、`$.get()`、`$.post()` 的底层实现，就是 `XMLHttpRequest` 。
- 历史
  - XMLHttpRequest对象最初是 [WHATWG(超文本应用程序技术工作组)](https://developer.mozilla.org/zh-CN/docs/Glossary/WHATWG) 的一部分。
  - 2006年移至W3C。
  - 2008年2月，对XMLHttpRequest进行了扩展（如进度事件和跨域请求），也就是所谓的 [XMLHttpRequest Level 2](http://dev.w3.org/2006/webapi/XMLHttpRequest-2/)，直至2011年底。
  - 2012年底，它移回了 [WHATWG](https://developer.mozilla.org/zh-CN/docs/Glossary/WHATWG)。
- 小结
  - XMLHttpRequest 简称 XHR 对象，是一个浏览器内置对象，目前，所有的浏览器均支持这个对象。
  - 它的作用是可以实现Ajax请求，Ajax技术的核心就是`XMLHttpRequest` 对象。

- **实现Ajax的GET请求**
  - 步骤
    - 创建xhr对象。
    - 注册 xhr.onreadystatechange 事件，当Ajax请求成功后，会触发onload函数。在 readystatechange 函数中，接收响应结果。
    - 调用open方法，初始化一个请求，此方法用于配置请求方式和url。
    - 调用send方法，发送请求。
  - 基础代码

```
// 1. 创建xhr对象
var xhr = new XMLHttpRequest();

// 2. 注册 xhr.onload事件，当Ajax请求成功后，会触发onload事件。在onload事件回调函数中，接收响应结果。
xhr.onload= function () {
    // 使用 xhr.response 接收响应结果
    var res = xhr.responseText;
}

// 3. 调用open方法，设置请求方式及请求的url地址
// xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks');
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks?id=1&bookname=西游记');

// 4. 最后，调用send方法，发送这次ajax请求
```

```
xhr.send();
```

        - 请求参数
            - 重要：如果传递请求参数，请求参数要以查询字符串形式拼接到url后面。
            - 形如：`url?id=1&bookname=西游记`
        - 其他说明

API	兼容性
XMLHttpRequest	IE7+ 支持
open	所有浏览器均支持
send	所有浏览器均支持
onreadystatechange	Level 2 新增，IE9+ 支持
response	Level 2 新增，IE10+ 支持

            - 了解`XMLHttpRequest`对象所有API的兼容性，点击[这里](https://developer.mozilla.org/zh-CN/docs/Web/API/XMLHttpRequest)。

- **实现Ajax的POST请求**
    - 步骤
        - （比GET请求多了一行代码）
        - 创建xhr对象。
        - 注册 xhr.onreadystatechange 事件，当Ajax请求成功后，会触发onload函数。在 readystatechange 函数中，接收响应结果。
        - 调用open方法，初始化一个请求，此方法用于配置请求方式和url。
        - 调用setRequestHeader方法，设置请求头。
        - 调用send方法，发送请求。
    - 基础代码

```
// 1. 创建xhr对象
var xhr = new XMLHttpRequest();

// 2. 注册 xhr.onreadystatechange 事件，当Ajax请求成功后，会触发onload函数。在readystatechange 函数中，接收响应结果。
xhr.onreadystatechange = function () {
    // 使用 xhr.response 接收响应结果
    var res = xhr.responseText;
}
```

```

// 3. 调用open方法，设置请求方式及请求的url地址
xhr.open('POST', 'http://www.liulongbin.top:3006/api/addbook');

// 4. 调用setRequestHeader，设置请求头，目的是告知服务器以何种方式解析请求体
xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');

// 5. 最后，调用send方法，发送这次ajax请求
xhr.send('bookname=西游记&author=唐僧&publisher=大唐出版社');
```

- 请求体
  - 重申一下，POST请求的参数，称之为请求体。
  - 和GET请求一样，请求体也要写成查询字符串格式，不同之处是，请求体做为send方法的参数发送。
- 其他说明

API	兼容性
setRequestHeader	所有浏览器均支持

  - `setRequestHeader` 方法用于设置请求头，格式如下：

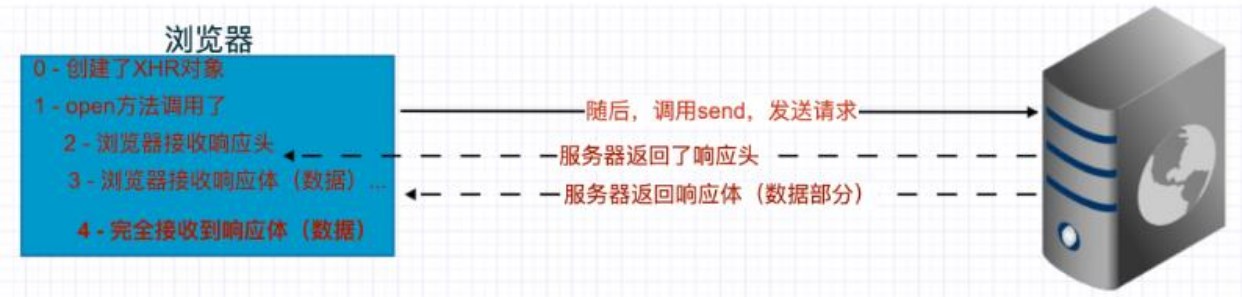
```
xhr.setRequestHeader('名称', '值');
```

    - 设置的请求头可以在 network 面板 > Headers > Request Headers 中查看。
    - 大部分请求头由浏览器管理，不允许我们修改。所以我们无需关心。
    - 一旦设置了请求头，就无法撤销了。
- Content-Type
  - 这里设置的 `Content-Type`，作用是告知服务器，浏览器提交的数据是何种类型的：
    - 值为：application/x-www-form-urlencoded（需要自己指定），表示客户端提交的是查询字符串。
    - 值为：application/json（需要自己指定），表示客户端提交的是 JSON 字符串。
    - 值为：multipart/form-data（xhr对象会自动设置），表示客户端提交的是 FormData 对象。

- **对请求参数进行编码**
  - 什么是URL编码?
    - 把中文和部分特殊符号转成 URL的标准格式，这就是url编码
    - 如，把“中文” 进行url编码后得到 “%E4%B8%AD%E6%96%87”
  - 为什么要对请求参数进行编码
    - RFC文档规定，只有字母和数字[0-9a-zA-Z]、一些特殊符号“$-_.+!*’(),”[不包括双引号]、以及某些保留字，才可以不经过编码直接用于URL。

        - 对于Unicode字符，RFC文档建议使用utf-8对其进行编码得到相应的字节，然后对每个字节执行百分号编码，也就是所谓的 URL编码。
    - 编码能够解决的问题
        - 乱码问题（浏览器使用utf-8、服务器使用其他编码，传输数据的时候，就会乱码）
        - 提交的内容有特殊符号问题，比如添加一本书，书名是 “红&黑”
    - 如何进行URL编码
        - JS提供了内置编码解码函数
            - 编码：encodeURIComponent('西游记')
            - 解码：decodeURIComponent("%E8%A5%BF%E6%B8%B8%E8%AE%B0");
        - 具体查看手册：https://www.w3school.com.cn/jsref/jsref_obj_global.asp
    - 小结
        - 无论是GET请求还是POST请求，向服务器传递参数的时候，必须要对参数进行编码。这里的参数包括参数名和值。比如，参数为 `shu ming=红&黑`
            - 参数名中有空格，需要编码；
            - 值有中文，且有歧义，需要编码；

- **readyState属性**
    - Ajax从创建xhr对象开始，一直到完全接收服务器返回的结果为止；我们可以把整个请求响应过程划分为5个阶段。并且可以使用 xhr.readyState 属性检测当前请求执行到哪个阶段了。
    - readyState属性值为一个数字，不同的数字表示Ajax的不同状态。
        - 如果状态值为0（xhr.readyState === 0），初始状态，表示xhr对象一定创建了。
        - 如果状态值为1（xhr.readyState === 1），表示open一定调用了
        - 如果状态值为2（xhr.readyState === 2），表示send一定调用了，并且已经接收到响应头。
        - 如果状态值为3（xhr.readyState === 3），表示正在接收服务器返回的数据（可能已接收完毕，也可能正在接收中，取决于数据量的大小）
        - 如果状态值为`4`（`xhr.readyState === 4`），表示Ajax请求~响应过程完成



    - 下面在创建xhr对象后和onreadystatechange事件内部分别输出 xhr.readyState 可以分别得到 0 、1、2、3 4。

```
// 1. 创建xhr对象
var xhr = new XMLHttpRequest();

console.log(xhr.readyState); // ===> 0
```

```

// 2. 注册 xhr.onreadystatechange 事件，当Ajax请求成功后，会触发onload函数。在readystatechange 函数中，接收响应结果。
xhr.onreadystatechange = function () {
    console.log(xhr.readyState); // ===> 4
    // 使用 xhr.response 接收响应结果
    var res = xhr.responseText;
}

// 3. 调用open方法，设置请求方式及请求的url地址
// xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks');
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks?id=1&bookname=西游记');

// 4. 最后，调用send方法，发送这次ajax请求
xhr.send();
```

- **onreadystatechange事件**
  - 介绍
    - onreadystatechange 翻译过来是 当Ajax的请求状态改变的时候。
    - 所以，它是配合上述的 readyState 使用的事件。
    - 事件具体的触发时机如下：
      - readyState属性值改变的时候
        - 0 --> 1
        - 1 --> 2
        - 2 --> 3
        - 3 --> 4
      - 接收到的数据量改变的时候，此时 readyState 的值保持为3，但也会触发 onreadystatechange 事件（发生在分块接收大量数据的时候）
  - 完整代码
    - 前文提到，onload有浏览器兼容问题，如果你的项目需要支持低版本的浏览器，那么可以使用onreadystatechange事件代替onload事件。
    - 由于onreadystatechange事件可能会触发多次，所以需要在事件中加入判断，已保证准确的接收到响应结果。

```

// 1. 创建xhr对象
var xhr = new XMLHttpRequest();
// IE6 创建对象 var xhr = new ActiveXObject('Microsoft.XMLHTTP');

```

```
// 2. 使用onreadystatechange代替onload
xhr.onreadystatechange = function () {
    // 判断Ajax请求是否完成
    if(xhr.readyState === 4) {
        // 还要根据响应状态码判断，请求是否成功
        if (xhr.status === 200) {
            console.log(xhr.responseText);
        } else {
            console.log('请求失败')
        }
    }
}

// 3. 调用open方法，设置请求方式及请求的url地址
// xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks');
xhr.open('GET', 'http://www.liulongbin.top:3006/api/getbooks?id=1&bookname=西游记');

// 4. 最后，调用send方法，发送这次ajax请求
xhr.send();
```

- **封装Ajax**
  - 思路：
    - 想好了，最终封装成什么样的 --- 封装成和 $.ajax 一样的（简略版）
    - 我们希望和调用jQuery的$.ajax方法一样，来调用自己封装的函数，形式如下：

```
// 假设我们封装了一个 ajax 函数，调用方式如下
ajax({
    type: 'GET',
    url: 'xxxx',
    data: {},
    success: function (res) {
        // res 表示服务器响应的结果
    }
});
```

    - 封装函数 ajax，参数只有一个，是对象形式，参照上面的代码
    - 函数体

    - 基本的Ajax代码（基本的步骤写出来）
    - 判断GET和POST请求，分别写 open 和 send 方法
    - 处理请求参数，把对象形式的参数处理成查询字符串格式
    - 当ajax请求成功之后，调用success函数，把服务器响应的结果当做实参传给success。
    - 细节处理（默认GET方式、不区分大小写等等、响应结果是否转换成JS对象...）
  - 参考代码如下：

```
/**
* 把字面量对象转换成查询字符串
* @param {object} obj 一个字面量对象
* @returns {string} 查询字符串
*/
function ObjectToQueryString(obj) {
    var arr = [];
    for (var key in obj) {
        arr.push(`${key}=${obj[key]}`);
    }
    return arr.join('&');
}
/**
* 实现Ajax请求
* @param {object} option 对象形式的参数
*   @param {string} option.type 请求方式
*   @param {string} option.url 接口地址
*   @param {object} option.data 请求参数
*   @param {callback} option.success 成功后的回调
*/
function ajax(option) {
    // 1. 把请求方式统一转大写，为后面的判断做准备
    var type = option.type.toUpperCase();
    // 2. 调用上面的函数，把请求参数处理成查询字符串
    var params = ObjectToQueryString(option.data);
    // 3. 写Ajax的基本步骤
    var xhr = new XMLHttpRequest();
    xhr.onreadystatechange = function () {
      if (this.readyState === 4 && this.status === 200) {
        // 当响应成功后，把JSON格式的结果转换为JS对象
        var res = JSON.parse(this.responseText);
        // 把结果，传递给 success 回调函数
        option.success(res);
```

```
      }
    }
    // 判断是GET还是POST请求方式
    if (type === 'GET') {
        xhr.open('GET', option.url + '?' + params);
        xhr.send();
    } else if (type === 'POST') {
        xhr.open('POST', option.url);
        xhr.setRequestHeader('Content-Type', 'application/x-www-form-urlencoded');
        xhr.send(params);
    }
}
```